

AI FitTrack Project Development

Phase 5

This implementation document explains the modules, dependencies, API behavior and source-level evidence present in the uploaded backend ZIP.

Field	Value
Project	AI FitTrack
Submission	SmartBridge Naan Mudhalvan
Team ID	SWTID-2026-8100
Team size	4
College	DRBCCC Hindu College
Department	B.Sc. Computer Science with Artificial Intelligence
Academic Year	2024–2027
Faculty Guide	Prof. D Saraswathy
Submission Date	
Team Leader	Parthiban M (NM ID: 28538B1DB05060E8D804CDAA88169909, Reg: 222401226)
Team Members	Naveen J (NM ID: 53C2DC1753C6DFDAF8649841B4630C80, Reg: 222401223); Nidesh N S (NM ID: FDE55A8DA9135F6AB7347C01AD1B8A17, Reg: 222401224); Nithish Kumar P (NM ID: BFBD9AD8CEC72229F374877DB82CC0B8, Reg: 222401225)
Project scope	Backend-only REST API
Technology	Node.js, Express.js, MongoDB, Mongoose, JWT and Google Gemini AI
Source baseline	Latest uploaded backend ZIP and supplied verification evidence
Document status	Submission-ready verified technical documentation with complete academic details

Prepared for academic evaluation. Secrets, credentials, private environment files and unverified claims are excluded.

Document contents

1. Implementation baseline
2. Module map
3. Setup and execution
4. Authentication
5. Workout module
6. Gemini module
7. Implementation evidence

Implementation baseline

The uploaded backend contains a root README, server package, dependency lockfile, safe environment example, models, routes, controllers, middleware, services, validation utilities, automated tests and a Postman collection. The archive contains no private .env file.

Runtime dependency	Uploaded version range	Purpose
express	^5.2.1	HTTP routing and middleware
mongoose	^9.10.1	MongoDB object-document mapping
jsonwebtoken	^9.0.3	JWT signing and verification
bcryptjs	^3.0.3	Password hashing and comparison
@google/genai	^2.24.0	Gemini content generation
cors	^2.8.6	Configurable cross-origin access
dotenv	^18.0.2	Local environment loading
supertest	^7.2.2	HTTP integration tests
mongodb-memory-server	^11.3.0	Isolated automated MongoDB testing

Implemented server module map

File groups present in the uploaded backend ZIP

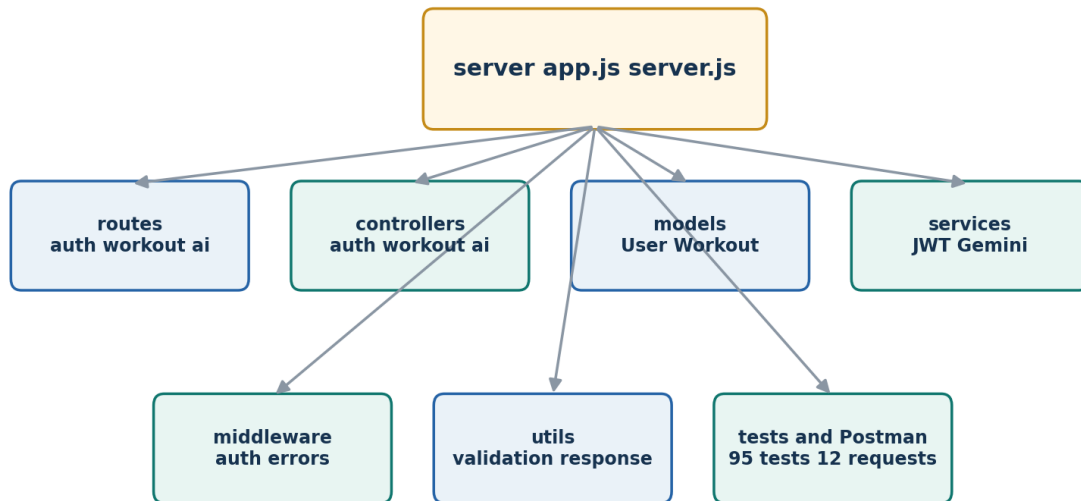


Figure 1 Uploaded server modules and verification assets

Setup and execution

1. Install Node.js 20 or later and MongoDB access.
2. Open the server folder and run `npm install` or `npm ci`.
3. Copy `.env.example` to `.env` and enter private values locally. Never commit that file.
4. Run `npm start` for the production-style start command or `npm run dev` for nodemon.
5. Confirm `GET /api/health` returns HTTP 200 before testing feature routes.

```

cd server
npm ci
# create server/.env locally from .env.example
npm start
# health: GET http://localhost:5000/api/health
  
```

Authentication implementation

Registration validates and normalizes input, checks for an existing email, creates a User document and returns a signed token with a safe user object. Login explicitly selects the stored password hash only for comparison and uses the same generic unauthorized message for an unknown email or incorrect password.

```

// authMiddleware.js - verified request identity
decoded = verifyToken(token);
const user = await User.findById(decoded.sub || decoded.id);
req.user = user;
req.userId = user._id.toString();
  
```

Workout implementation

All workout routes apply protect before the controller. Create assigns the owner from req.user._id. Read, update and delete operations combine the workout identifier with the authenticated owner. Search escapes user-entered regular-expression characters, canonicalizes category values and uses explicit UTC day boundaries.

```
// workoutController.js - ownership is part of the database query
const workout = await Workout.findOne({
  _id: id,
  user: req.user._id
});
```

Gemini implementation

The AI controller validates inputs and enforces a per-user, per-process request limit. The service requests structured JSON from Gemini, applies a per-attempt timeout, retries only transient failures, respects Retry-After within a bounded delay and converts provider failures into safe application errors.

Endpoint	Input	Structured output
workout-recommendation	age, fitnessGoal, experienceLevel or experience	Plan, weeklySchedule, experience guidance, tips, recovery, safety and motivation
fitness-insights submitted	totalWorkouts, averageWorkoutDuration or averageDuration, totalCaloriesBurned	Performance, consistency, improvement, motivation and progress summary
fitness-insights database	useDatabaseStats true	Same insight structure using only the authenticated user's MongoDB records

API implementation catalog

Method	Endpoint	Access	Purpose	Main statuses
GET	/api/health	Public	Service health and runtime metadata	200
POST	/api/auth/register	Public	Create user and issue JWT	201, 400, 409
POST	/api/auth/login	Public	Verify credentials and issue JWT	200, 400, 401
GET	/api/auth/email-available	Public	Check normalized email availability	200, 400
GET	/api/auth/profile	JWT	Return authenticated user profile	200, 401
POST	/api/workouts	JWT	Create an owner-scoped workout	201, 400, 401
GET	/api/workouts	JWT	List history and optional filters	200, 400, 401
GET	/api/workouts/search	JWT	Search by name, category or UTC date	200, 400, 401

Method	Endpoint	Access	Purpose	Main statuses
GET	/api/workouts/:id	JWT	Read one owned workout	200, 400, 404
PUT	/api/workouts/:id	JWT	Update allowed workout fields	200, 400, 404
DELETE	/api/workouts/:id	JWT	Delete one owned workout	200, 400, 404
POST	/api/ai/workout-recommendation	JWT	Generate structured workout plan	200, 400, 429, 5xx
POST	/api/ai/fitness-insights	JWT	Generate submitted or database-derived insights	200, 400, 429, 5xx

Source-to-feature evidence

Feature	Primary source files	Evidence basis
Server lifecycle	server.js, config/db.js	Connect before listen; graceful shutdown; meaningful connection errors.
API gateway	app.js	CORS, body parsing, health, route mounting, 404 and centralized errors.
Auth	authRoutes.js, authController.js, User.js, jwtService.js, authMiddleware.js	Register, login, profile and missing-token live records.
Workouts	workoutRoutes.js, workoutController.js, Workout.js, workoutValidation.js	Eight live CRUD/search records and 12 workout tests.
Gemini	aiRoutes.js, aiController.js, geminiService.js, aiValidation.js	Three live AI records and AI/retry automated tests.
Standard errors	ApiError.js, errorMiddleware.js, response.js	400, 401, 404, 409 and AI error tests.
Manual API run	postman/AI-FitTrack.postman_collection.json	12 ordered requests across health, auth, workouts and AI.

Implementation evidence limits

- The supplied evidence contains sanitized text captures rather than evaluator-facing screenshots.
- The evidence records an earlier delete response with data:null, while the uploaded controller currently returns the deleted workout object inside data.workout. The delete behavior is verified, but the exact current payload should be re-captured.
- The submitted-statistics evidence labels statsSource as manual, while the uploaded controller sets submitted. The AI behavior is verified, but that response label should be re-captured from the frozen ZIP.
- The current workspace did not reinstall dependencies, so no new test outcome is claimed here; all pass counts are taken from the supplied verification records.

Items requiring student confirmation

Pending item	What to provide
Repository	Insert the final public GitHub repository URL after upload.
Deployment	Insert the deployed API base URL if the evaluator requires a hosted demonstration.